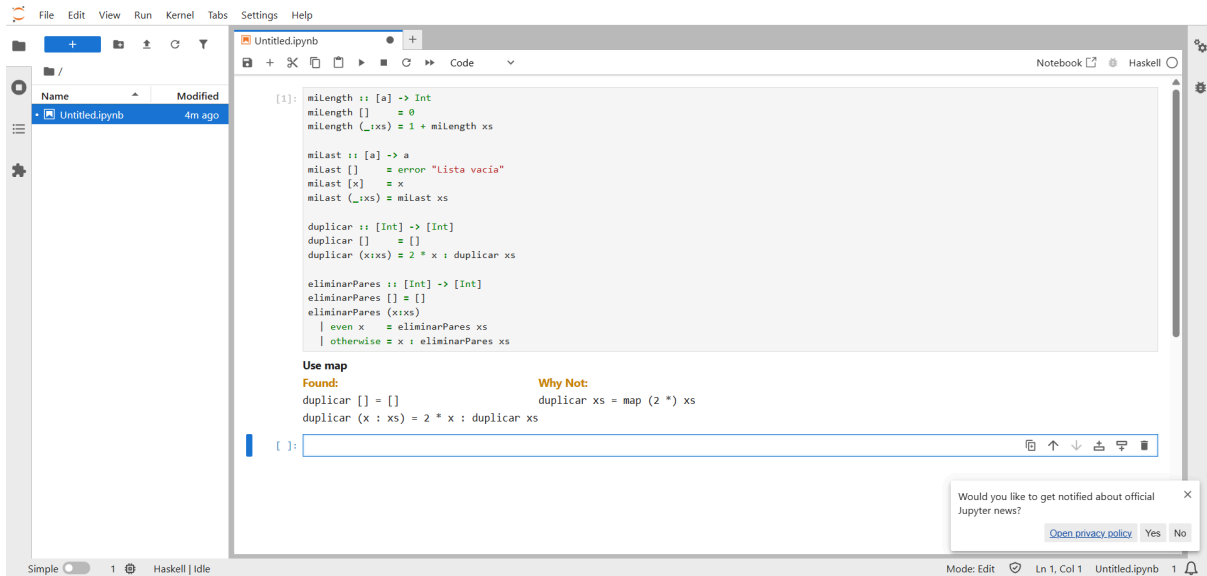


Ejercicio 1)



The screenshot shows a Jupyter Notebook titled 'Untitled.ipynb' in the 'Code' view. The code defines several functions for list manipulation in Haskell:

```
[1]: milength :: [a] -> Int
      milength [] = 0
      milength (_:xs) = 1 + milength xs

      milast :: [a] -> a
      milast [] = error "Lista vacia"
      milast [x] = x
      milast (_:xs) = milast xs

      duplicar :: [Int] -> [Int]
      duplicar [] = []
      duplicar (x:xs) = 2 * x : duplicar xs

      eliminarPares :: [Int] -> [Int]
      eliminarPares [] = []
      eliminarPares (x:xs)
        | even x = eliminarPares xs
        | otherwise = x : eliminarPares xs
```

Below the code, there is a section titled 'Use map' with a 'Found:' note and a 'Why Not:' note. The 'Found:' note shows the current implementation of `duplicar` using a list comprehension. The 'Why Not:' note suggests using `map` to achieve the same result more concisely.

```
Found:
duplicar [] = []
duplicar (x : xs) = 2 * x : duplicar xs

Why Not:
duplicar xs = map (2 *) xs
```

At the bottom of the code cell, there is an input prompt `[ ]:` followed by a text box for user input.

A notification dialog box is visible in the bottom right corner, asking 'Would you like to get notified about official Jupyter news?' with 'Yes' and 'No' buttons and a link to the 'Open privacy policy'.

Ejercicio 2)



The screenshot shows a Jupyter Notebook titled 'Untitled.ipynb' in the 'Code' view. The code defines several functions for list manipulation in Haskell, including `milength'`, `milast'`, `duplicar'`, and `eliminarPares'`.

```
[2]: milength' :: [a] -> Int
      milength' = foldr (\_ acc -> acc + 1) 0

      milast' :: [a] -> a
      milast' = foldr1 (\_ x -> x)

      duplicar' :: [Int] -> [Int]
      duplicar' = map (*2)

      eliminarPares' :: [Int] -> [Int]
      eliminarPares' = filter odd
```

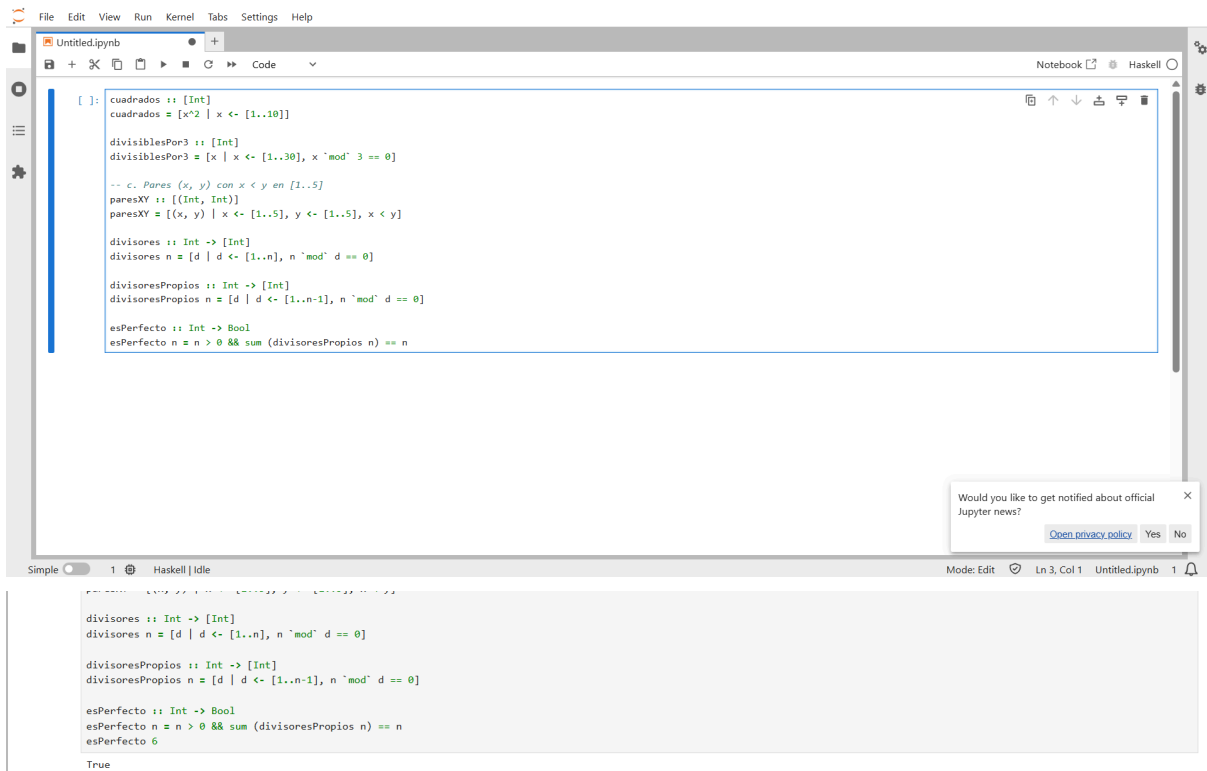
Below the code, there is a section titled 'Use map' with a 'Found:' note and a 'Why Not:' note. The 'Found:' note shows the current implementation of `duplicar` using a list comprehension. The 'Why Not:' note suggests using `map` to achieve the same result more concisely.

```
Found:
duplicar [] = []
duplicar (x : xs) = 2 * x : duplicar xs

Why Not:
duplicar xs = map (2 *) xs
```

At the bottom of the code cell, there is an input prompt `[ ]:` followed by a text box for user input.

A notification dialog box is visible in the bottom right corner, asking 'Would you like to get notified about official Jupyter news?' with 'Yes' and 'No' buttons and a link to the 'Open privacy policy'.



```
[ ]: cuadrados :: [Int]
cuadrados = [x^2 | x <- [1..10]]

divisiblesPor3 :: [Int]
divisiblesPor3 = [x | x <- [1..30], x `mod` 3 == 0]

-- c. Pares (x, y) con x < y en [1..5]
paresXY :: [(Int, Int)]
paresXY = [(x, y) | x <- [1..5], y <- [1..5], x < y]

divisores :: Int -> [Int]
divisores n = [d | d <- [1..n], n `mod` d == 0]

divisoresPropios :: Int -> [Int]
divisoresPropios n = [d | d <- [1..n-1], n `mod` d == 0]

esPerfecto :: Int -> Bool
esPerfecto n = n > 0 && sum (divisoresPropios n) == n
```

Would you like to get notified about official Jupyter news?

[Open privacy policy](#) Yes No

```
divisores :: Int -> [Int]
divisores n = [d | d <- [1..n], n `mod` d == 0]

divisoresPropios :: Int -> [Int]
divisoresPropios n = [d | d <- [1..n-1], n `mod` d == 0]

esPerfecto :: Int -> Bool
esPerfecto n = n > 0 && sum (divisoresPropios n) == n
esPerfecto 6
True
```

Ejercicio 3)

```
File Edit View Run Kernel Tabs Settings Help
Untitled.ipynb
[ ]: import Data.Char (isUpper, toLower)

-- 1
sumalista :: [Int] -> Int
sumalista = foldr (+) 0

mayoresQue10 :: [Int] -> Int
mayoresQue10 = length . filter (> 10)

productolista :: [Int] -> Int
productolista = foldr (*) 1

celsiusAFahrenheit :: [Double] -> [Double]
celsiusAFahrenheit = map (\c -> c * 9 / 5 + 32)

ajustarImpares :: [Int] -> [Int]
ajustarImpares = map (\x -> if odd x then x + 5 else x)

-- 2
nombresLargos :: [String] -> [String]
nombresLargos = filter ((> 5) . length)

soloNegativos :: [Int] -> [Int]
soloNegativos = filter (< 0)

edadesUniversitarios :: [Int] -> [Int]
edadesUniversitarios = filter (\e -> e >= 18 && e <= 25)

-- 3
agregarIVA :: [Double] -> [Double]
agregarIVA = map (* 1.19) . filter (>= 1000)

empiezaConVocal :: String -> Bool
empiezaConVocal [] = False
empiezaConVocal (c:_) = toLower c `elem` "aeiouáéíóú"

longitudesVocales :: [String] -> [Int]
longitudesVocales = map length . filter empiezaConVocal

cuadradosPares :: [Int] -> [Int]
cuadradosPares = map (^2) . filter even

-- 4
hayNegativo :: [Int] -> Bool
hayNegativo = any (< 0)

todosPares :: [Int] -> Bool
todosPares = all even

nombresCorrectos :: [String] -> Bool
nombresCorrectos = all (\n -> case n of
    (c:_) -> isUpper c
    [] -> False)

-- 5
menoresQue100 :: [Int] -> [Int]
menoresQue100 = takeWhile (< 100)

sinCerosIniciales :: [Int] -> [Int]
sinCerosIniciales = dropWhile (== 0)

sinCerosIniciales = dropWhile (== 0)

[3]: agregarIVA' :: [Double] -> [Double]
agregarIVA' = map (\p -> if p >= 1000 then p * 1.19 else p)
```

Ejercicio 4)

```
[4]: procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
procesarLecturas f xs = map f xs

-- Conversión de un solo valor (el README la llama celsiusAFahrenheit)
celsiusAF :: Double -> Double
celsiusAF c = c * 9 / 5 + 32

crearFiltro :: (a -> Bool) -> ([a] -> [a])
crearFiltro p = filter p

enRango :: Int -> Int -> (Int -> Bool)
enRango a b = \x -> x >= a && x <= b

potenciador :: Int -> (Int -> Int)
potenciador n = \x -> x ^ n
procesarLecturas celsiusAF [0, 12, 20] -- [32.0,53.6,68.0]

soloPares :: [Int] -> [Int]
soloPares = crearFiltro even
soloPares [1..10]
entre5y10 :: Int -> Bool
entre5y10 = enRango 5 10
entre5y10 7

potencia5 :: Int -> Int
potencia5 = potenciador 5
potencia5 7
```

Eta reduce

Eta reduce

Found: **Why Not:**

procesarLecturas f xs = map f xs
procesarLecturas = map f xs

Eta reduce

Found: **Why Not:**

crearFiltro p = filter p
crearFiltro = filter

Redundant lambda

Found: **Why Not:**

enRango a b = \ x -> x >= a && x <= b
enRango a b x = x >= a && x <= b

Redundant lambda

Found: **Why Not:**

potenciador n = \ x -> x ^ n
potenciador n x = x ^ n

Avoid lambda using 'infix'

Found: **Why Not:**

\ x -> x ^ n(^ n)

[32.0,53.6,68.0]

[2,4,6,8,10]

True

16807

Ejercicio 5)

```
File Edit View Run Kernel Tabs Settings Help

Untitled.ipynb

Name Modified
Untitled.ipynb now

[ ]: import Data.List (sortOn)

type Estado = (Int, Int)
type Nodo   = Estado
type Costo  = Int
type Grafo  = [(Nodo, [(Nodo, Costo)])]

capA, capB :: Int
capA = 5
capB = 3

transferirAB :: Estado -> Estado
transferirAB (a, b) = (a - t, b + t)
  where t = min a (capB - b)

transferirBA :: Estado -> Estado
transferirBA (a, b) = (a + t, b - t)
  where t = min b (capA - a)

vecinos :: Estado -> [(Estado, Costo)]
vecinos (a, b) =
  [ ((capA, b), 1)
  , ((a, capB), 1)
  , ((0, b), 1)
  , ((a, 0), 1)
  , (transferirAB (a, b), 1)
  , (transferirBA (a, b), 1)
  ]
```

```
]

heuristic :: Estado -> Costo
heuristic (4, 0) = 0
heuristic _     = 1

aStar :: (Nodo -> [(Nodo, Costo)]) -> (Nodo -> Costo) -> Nodo -> Nodo -> [(Nodo, Costo)]
aStar sucesores heuristica inicio meta = buscar [(inicio, 0)] []
  where
    buscar [] _ = []
    buscar ((nodo, costo):cola) visitados
      | nodo == meta = [(nodo, costo)]
      | nodo `elem` visitados = buscar cola visitados
      | otherwise =
        let nuevos = [ (v, costo + c) | (v, c) <- sucesores nodo, v `notElem` visitados ]
            ordenados = sortOn (\(v, c) -> c + heuristica v) (cola ++ nuevos)
        in (nodo, costo) : buscar ordenados (nodo : visitados)
```

```
aStar vecinos heuristic (0, 0) (4, 0)
[ ((0,0),0),((5,0),1),((0,3),1),((5,3),2),((2,3),2),((3,0),2),((2,0),3),((3,3),3),((0,2),4),((5,1),4),((5,2),5),((0,1),5),((4,3),6),((1,0),6),((4,0),7)]
```